Constructing Computation Sources

A smaller complete iPRA checker and a finite-range obstruction

Research prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

We give an explicit proof presentation of full first-order intuitionistic primitive recursive arithmetic, with a standalone checker strictly below the observed complete-checker minima in the nine current ProveIt draft archives. Under the unchanged whole-program normalizer, the selected checker has **279 tokens, 314 syntax-tree nodes, and 682 bytes**, compared with the current coordinate envelope **281/315/685**.

The mathematical change is small and precise. A computation certificate supplies a predecessor and a base/step flag; the checker constructs the source counter. The predecessor image is checked only if the update uses it. This replaces an eager validation obligation by a construction invariant and a state-independence argument. The change preserves the native theory and its whole-domain bi-interpretation with iPRA. An explicitly attributed appendix reproduces the full inherited arithmetic and logical bridge.

A separate new result settles a current-draft question negatively: restricting paired-introduction witnesses to zero or variables and closure-specialization targets to one or variables loses native theorems. Adding any fixed finite set of numeral targets does not repair the loss.

These are a reproducible implementation reduction and a mathematical obstruction theorem. They do not establish an absolute simplest axiomatization or the significant breakthrough requested. The source-size gain is modest; the arithmetic basis and the logical reconstruction are inherited. The proofs are written mathematical arguments, not a Lean or Rocq verification.

Contents

1 The precise improvement **3**

2 The complete native axiomatization **4**

3 The new computation-certificate principle **5**

4 Recovering logic and arithmetic **7**

5 The entire selected checker **9**

6 A new obstruction to simultaneous restrictions **10**

7 Verification, reproduction, and limits **11**

A Computation sources certified by construction **12**

B A finite-range obstruction to simultaneous witness and specialization restriction **14**

The inherited bridge, reproduced with attribution **16**

C Complementary introductions with one local assumption **16**

D Internal scalar predecessor bootstrap **21**

E From literal-equation induction to full intuitionistic PRA **25**

F Pairing, primitive recursion, and internal numerical composites **31**

G Arithmetic strength and both ordinary interpretation composites **32**

1 The precise improvement

The comparison is pinned to ProveIt commit

`4a04139595eadc3e8818f1df2a94838ed4851d83`.

The `docs/drafts` tree is `cec0cd82d69646386c802841068c17ff76d188fc`. It contains nine direct ZIP archives[1]. Their exact bytes, Git blob hashes, SHA-256 hashes, and member inventories are included in the package. The source scan measured 261 extracted Python occurrences with 164 distinct contents. An additional scan of ten nested provenance archives measured 154 historical occurrences, with 109 distinct contents; none improves the current complete-checker envelope.

Files smaller than the envelope were inspected before classification: build, measurement, experiment, and verification scripts are not complete theorem checkers. Complete historical checkers inside reference folders do count. The comparison is with the present combined corpus, not the older minima quoted by individual reports.

Entire standalone program	Tokens	AST nodes	Bytes
Generated assumptions, principal[2]	281	318	685
Generated assumptions, AST profile[2]	283	315	710
Current coordinate envelope	281	315	685
Constructed computation, explicit assignments	277	317	684
Constructed computation, AST profile	279	314	709
Constructed computation, selected profile	**279**	**314**	**682**
Old generator with the same initialization profile	283	315	683

The selected single program strictly improves all three current minima: two fewer tokens, one fewer AST node, and three fewer bytes. The percentage reductions against those separate minima are approximately 0.71%, 0.32%, and 0.44%. They are small. The three minima are attained by different old programs, whereas the selected new program beats all three simultaneously.

The arithmetic generator itself changes the explicit program from 281/318/685 to 277/317/684. The selected profile additionally folds initialization into an existing default expression and folds the input binding into its first call. Those two Python idioms are inherited source-layout techniques, not new mathematical principles. Applying the identical profile to the old generator yields 283/315/683: the genuine generator comparison remains four tokens, one AST node, and one byte. The table separates these effects so that the improvement cannot be misattributed to a new arithmetic foundation.

1.1 What the metric includes

The unchanged normalizer removes genuine docstrings, parses the complete module, applies `ast.unparse` followed by a newline, and counts significant tokens, all AST nodes, and UTF-8 bytes. Measurements use CPython 3.12.14. Initial state, JSON input, traversal, scope checks, inference dispatch, and the complete ACCEPT/REJECT driver are included. Every counted checker imports only the standard JSON library. Producers, converters, and the numerical testing decoder are not trusted by it.

The Python runtime and standard library are the shared platform, as in the drafts. No interpreter, axiom table, theorem-specific library, or unchecked executable certificate is moved outside the counted file. The selected and old programs have the same outer JSON container, four-field packet arity,

and verdict convention. Computation packets use the new predecessor/mode semantics and require translation. In particular, rejected inputs print `REJECT`, exit nonzero, and may produce a traceback; accepted inputs print `ACCEPT` and exit zero. The program does not use assertions whose behavior changes under Python optimization.

The observed sizes are upper bounds on an implementation optimum. They are not mathematical lower bounds for all theories or all programming languages. This report meets the operational comparison with the supplied complete programs; it does not prove global minimality or claim that a one-byte algorithmic reduction constitutes a major simplification of arithmetic.

2 The complete native axiomatization

The target is *full first-order* iPRA: intuitionistic logic with actual numerical equality, genuine falsity, function symbols for fixed finite primitive-recursive descriptions, their defining equations, the usual successor boundary laws, and quantifier-free induction. Atomic-induction presentations of iPRA also occur in the literature[8]; here the reduction from quantifier-free induction is proved internally in the reproduced bridge. No equivalence with a merely quantifier-free theorem set is substituted for bi-interpretability.

2.1 Numerical and formula syntax

There is one numerical sort. The numerical terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only h binds v; n, a do not. Other parameters of h remain fixed. Each body is a fixed finite earlier description. Equivalently, at each external syntactic stage introduce a first-order symbol $i_h(n, a, \vec{p})$ for each earlier term $h(v, \vec{p})$. There is no function quantifier and no uniformly total evaluator for all primitive-recursive descriptions.

The two computation patterns are

$$I(0, a; v.h) = a, \tag{I0}$$
$$I(Sn, a; v.h) = h[I(n, a; v.h)/v]. \tag{IS}$$

The formula grammar is

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A,$$

with distinct paired bound variables. A context is a finite reusable list of these formulas. Substitution is capture-free; binders are renamed before substituting. The designated absurdity is $\bot_* = (0 = S0)$. The ordinary readings, whose adequacy will be proved rather than assumed, are

$$(Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$
$$(D_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

2.2 Every primitive rule

The presentation $\mathcal{K}$ has the following complete inventory. Hypothesis use, equality transport, and closure use are

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}, \qquad \frac{\Gamma \vdash Q_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \qquad \text{(Hyp, Tr, QE)}$$

The motive F is any fixed native formula template. Transport does not license induction on F.

Paired elimination is

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, Q_yB(a,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

Here a is absent from Γ, A and the fixed parameters of B. Paired introduction has two forms:

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)}. \tag{DI-case, DI-progress}$$

The second matrix E must be a *literal equation*. A formula with a Q or D at its root is not literal. In both forms z is absent from Γ, t and all fixed matrix parameters. Displayed local hypotheses are discharged. Case introduction is allowed for every native matrix.

Finally, numerical computation is supplied as a temporary assumption:

$$\frac{\Gamma, E \vdash A}{\Gamma \vdash A} \quad (E \text{ is an instance of I0 or IS}). \tag{Comp}$$

This is two finitely specified computation schemas, whose instances are arbitrary finite terms with the stated bindings. There are no additional logical or nonlogical axioms, primitive reflexivity rules, coverage axioms, separation axioms, unrestricted induction rules, or hidden first-order proof engines.

The reference calculus $\mathcal{C}$ instead has I0 and IS as theorem leaves and the same five logical rule families. Both presentations prove exactly the same native sequents: Comp is cut with a computation theorem, and a computation theorem is obtained in $\mathcal{K}$ by Comp followed by Hyp. Weakening, exchange, contraction, renaming, numerical substitution, and cut are admissible by induction on finite derivations. Rename local parameters before substitution or cut; substitution preserves the root constructor and hence the literal-equation induction restriction.

3 The new computation-certificate principle

The current leading draft supplies an alleged iterator source $[c, a, h]$. In a successor step it expands c and eagerly checks the predecessor iterator before substituting it into h. This guards against a non-successor counter that happens to have a truthy serialization. The old check is important: simply deleting it admits a false equation.

The new certificate supplies the *predecessor* p, the initializer a, the update h, and an explicit mode e. The checker itself forms the source:

$$\begin{array}{l|c} e = 0 & I(0, a; v.h) = a \\ e = 1 & I(Sp, a; v.h) = h[I(p, a; v.h)/v]. \end{array}$$

The mode occupies an already existing, previously unused computation field. It does not lengthen the packet. The source has a genuine successor constructor because the checker constructs it. In particular, the successor-of-zero case uses $p = 0, e = 1$; there is no restriction to nonzero predecessors or any special zero-alias convention.

The two lines implementing the new generator are

```
n, t, r = s
h = [[u and [n], t, r], i(r, s, 1) if u else t]
```

Here $s = [p, a, h]$ and the packet's u is the mode. The local t, r become the initializer and update. Only canonical flags 0 and 1 are needed for completeness; other raw values are handled as described below.

3.1 Why validation can be delayed

The opener i checks a substituted numerical image whenever an exposed numerical coordinate actually uses that image. Therefore the old eager validation can be separated into two distinct obligations:

1. the source counter is a successor;
2. a recursive image used in the result is a valid numerical term.

Construction establishes the first. Demand-driven substitution establishes the second. If the update ignores its state, the recursive image is absent from the generated equation and needs no separate validation.

There is a further scope issue: the common continuation introduces a new parameter w, and a raw generated equation can depend on it. Its zero instance, rather than the equation at every w, is what must be proved.

Lemma 3.1 (Known-instance discharge)**.** *If $w \notin \mathrm{FV}(\Gamma, A)$, $\Gamma \vdash H(0)$, and $\Gamma, H(w) \vdash A$, then $\Gamma \vdash A$.*

Proof. Rename eigenparameters in the continuation to avoid capture. Substitute zero for w, leaving the retained context and goal unchanged. This gives $\Gamma, H(0) \vdash A$. Cut with the given proof of $H(0)$. □

Theorem 3.2 (Constructed computation supply)**.** *Every equation $H(w)$ successfully generated by the new arithmetic branch has a zero instance provable from I0 or IS, with equality substitution. Consequently its common continuation is admissible in $\mathcal{C}$ and hence in $\mathcal{K}$.*

Proof. In base mode, after setting $w = 0$, the source initializer and its copy on the right agree by structural zero completion. I0 gives the equation. The predecessor payload is unused and has no object-language meaning in this branch.

In step mode, first suppose that opening the update requests its recursive image. It then validates $[p, a, h]$ at numerical root. Successful numerical validation is idempotent, so doing it on demand gives the same substituted value as validating eagerly. The extra dummy zero slot beneath the constructed successor cannot introduce a meaningful new dependency: numerical-root validation of the predecessor image would reject that dependency. The two occurrences of the predecessor therefore agree, and the zero instance is IS.

If the update never requests its image, it is independent of the outer state on every meaningful numerical path. Genuine internal variables are protected; a meaningful outer-state reference would request the image. Numerical binary annotations are semantically zero and contribute no hidden state dependence. The common opening still checks the entire source iterator. After zero completion it has the form $I(Sp_0, a_0; v.h_0) = h_0$, where h_0 is independent of v. IS proves this equation without needing an unused recursive image.

The old goal and context were checked before allocating w, so they have no meaningful occurrence of it. Lemma 3.1 discharges the actual generated assumption. Appendix A gives the full raw-coordinate argument. □

Corollary 3.3 (No change to the native theory)**.** *The new certificate calculus proves exactly the closed native theorems of $\mathcal{K}$. It preserves the same sequents and the same interpretation composites, rather than merely preserving standard numerical truth.*

Proof. Soundness follows from Theorem 3.2 and the unchanged logical-rule checks. For completeness, encode each Comp instance with its mode and predecessor; supply the generated equation and close

its use by Hyp. Other instructions and their eigenparameter allocation are unchanged. Canonical numerical syntax uses rigid labels for passive parameters and zeros in dummy positions. Both computation equations, including the step with zero predecessor, are generated exactly. □

4 Recovering logic and arithmetic

This section summarizes the inherited construction; the complete arguments appear in Appendices C–G. Their provenance and unchanged source hashes are recorded. The new contribution is the conservative certificate generator above, not the following arithmetic bootstrap. The inherited logic appendix presents D formulas with auxiliary clauses. The present nested-Q syntax is covered because $Q_y B$ is interderivable with $D_{v,y}B$ for unused v: case DI gives one direction, and DE followed by Hyp gives the other. External induction eliminates nested Q occurrences, using the same uniform matrix-congruence argument. Thus no extra logical connective is left without a round trip.

Put $C(n,a,b) = I(n,a;v.b)$. Then $C(0,a,b) = a$ and $C(Sn,a,b) = b$. From computation and transport one derives reflexivity, symmetry, transitivity, and congruence. DI-progress, DE, and QE give contextual literal-equation induction. DI-case, DE, and QE give numerical cases for arbitrary native formulas. These constructions retain arbitrary assumptions, with the displayed eigenparameter conditions.

For a formula A, form a gate $G_z A$ recursively. At an equation leaf replace $s = t$ by $C(z,0,s) = C(z,0,t)$, and continue beneath binders. Then $G_0 A$ is provable and $G_{Sz}A$ is interderivable with A. Padding by a vacuous paired binder allows two formulas of different depths to be aligned. A recursively constructed selector $M_z(A,B)$ then satisfies

$$M_0(A,B) \dashv\vdash A, \qquad M_{Sz}(A,B) \dashv\vdash B.$$

The scalar parity term $p(z) = I(z,0;v.C(v,1,0))$ has $p(1) = 1, p(2) = 0$. With fresh bound variables, the following macros recover the full intuitionistic interface:

$$\begin{aligned} A \Rightarrow_* B &= D_{u,z} M_z(A,B), \\ A \wedge_* B &= D_{u,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &= D_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &= A(0) \wedge_* D_{u,z} G_z A(z), \\ \bot_* &= (0 = S0). \end{aligned}$$

The coordinate u is unused. The selector gives implication rules; the gate gives existential rules; numerical cases and selectors give disjunction; the two parity values give conjunction projections. Coverage gives universal elimination in arbitrary retained contexts. Under $0 = S0$, the selector and transport give every equation; structural formula recursion then gives explosion. No excluded-middle rule for quantified formulas is introduced.

The ordinary realization T_I has standard intuitionistic equality logic, I0 and IS, contextual literal-equation induction, and the explicit boundary sentences

$$\forall x(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The falsity bridge is necessary: positive numerical equations alone admit a one-element structure.

The arithmetic reconstruction has a fixed noncircular order. First an internal scalar predecessor is built using numerical cases and equation induction. Then truncated subtraction and a characteristic

of actual equality are constructed and proved correct for all source elements. Each fixed quantifier-free formula has a characteristic equation, so its induction follows. Finally pairing and projections are constructed and general primitive recursion is compiled. Pairing is not assumed in the predecessor argument.

If π, L, H are the proved pairing and projections, a general recursion $f(0,\vec{p}) = g(\vec{p})$, $f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p})$ is represented by

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); v.\pi(SL(v), h^*(L(v), H(v),\vec{p}))\big),$$
$$f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$, and the projection identities prove both defining equations. This is an external compiler for each finite description, not an internal uniformly total evaluator.

Theorem 4.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization of $\mathcal{K}$ is bi-interpretable and equiconsistent with full first-order iPRA. In each numerical composite, the definable isomorphism is $x = y$ on the entire domain.*

Proof. Interpret scalar iteration in iPRA by primitive recursion ignoring the counter in its update; interpret each iPRA description in T_I by the scalar compiler. The preceding internal arithmetic proves all source axioms and quantifier-free induction instances. The logical macros and ordinary readings preserve all formulas and proof rules.

For either numerical composite, the double translation satisfies the original defining equations. Literal-equation induction proves uniqueness with all passive parameters fixed. External induction on a fixed finite description therefore identifies the operation with its double translation at every internal argument. The identity graph is internally total, bijective, and operation-preserving. No quotient, definable cut, or restriction to standard numerals is used.

For formula composites, the decisive native equivalences are

$$Q_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y B(y),$$

$$D_{x,y} B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big).$$

DE proves the forward direction by exposing a witness and its closure; the reverse uses existential elimination and case DI, with its successor premise supplied by implication and universal elimination. Structural induction gives all formula round trips. The explicit falsity bridge preserves contradiction. Theorem 3.2 transfers the interpretations unchanged to the new certificate presentation. Finite proof translations yield equiconsistency, with no claim that either theory proves its own consistency. □

5 The entire selected checker

```
from json import load
i = lambda a, t=None, d=(k := 0): [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if isinstance(a,
     list) else a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t]) and i(t, u) == a:
            a = i(t, s)
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, r = s
        h = [[u and [n], t, r], i(r, s, 1) if u else t]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    c((q := load(open(0)).pop)(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The assignment to k in the lambda's default is evaluated once when the function is created at module scope; it does not reset freshness during calls. Likewise the input assignment initializes q before its first call. The explicit-assignment implementation has the same successful traces and is supplied for easier inspection. The descriptive reference implementation uses meaningful names and separates the branches. None of these files calls a proof producer or a theorem library.

5.1 Input, tree decoding, and scopes

Input is a JSON array. Its last item is the requested closed formula; the preceding four-field packets $[m, s, u, t]$ are popped from the end. Hypothesis closure consumes no packet. An unread prefix is inert, as in the comparison programs. A malformed, exhausted, or unsuccessful input rejects. Physical Python recursion or memory limits may reject large valid certificates; mathematical completeness concerns unbounded resources on finite canonical certificates.

Array length	Formula reading	Numerical reading	Depth added
0	truth $0 = 0$	zero	no children
1	Q	successor	2
2	equation	zero with checked annotations	1
3	D	scalar iterator	4

Larger arrays reject when traversed. Let ρ be the environment of protected numerical slots. The main decoder clauses are

$$\begin{aligned} T_\rho([p]) &= ST_{0,\rho}(p), & F_\rho([B]) &= Q_y F_{y,\rho}(B), \\ T_\rho([p,q]) &= 0, & F_\rho([s,t]) &= (T_\rho(s) = T_\rho(t)), \\ T_\rho([n,a,h]) &= I(T_{0,0,\rho}(n), T_{0,0,\rho}(a); v.T_{v,v,\rho}(h)), \\ F_\rho([P,U,B]) &= D_{x,y}F_{y,x,\rho}(B). \end{aligned}$$

The two padding fields of D and the annotations of a numerical binary node are checked, although they contribute no mathematical data. In particular the scalar iterator has two aliases for its one state, not an extra recursion-counter input.

At formula depth $2q$ and numerical depth $2q+1$ there are q protected slots. Positive coordinates $2j-1, 2j$ name the same jth slot, nearest first. Scalar zero denotes numerical zero; negative labels name already allocated rigid parameters. The floor k starts at zero and decreases at each continuation. The opener preserves labels in $[k,d)$, inserts a checked numerical image at an exposed numerical coordinate, and maps an exposed formula leaf to truth. Missing meaningful images reject. Canonical inputs use integers and arrays. Accepted finite fractional aliases have the inherited floor-coordinate meaning; noncanonical data do not add new numerical operations. Scope safety and the common-motive opening lemma are structural facts about this traversal.

5.2 All command branches

For nonempty m, the checker first proves the major m. Matched equality or closure endpoints and the target equality implement Tr or QE. They change the goal to the appropriate minor and add only truth to the context. An inapplicable Eq/Q request leaves the goal unchanged and adds only truth. A three-field D major adds its opened Q closure, giving DE. An unsuccessful major never falls through as if it had succeeded.

For false m and true t, the command is DI with witness t. Canonical marker $m = []$ selects the zero-case assumption; marker $m = 0$ selects the current hypothesis only when the matrix is a literal equation, and otherwise selects the zero case. A fresh successor is the new goal. Thus arbitrary compound induction is not silently introduced.

For false m and false t, the new arithmetic generator uses $s = [p,a,h]$ and mode u. It constructs the source and supplies the generated equation. Theorems above justify all successful raw cases, including those whose generated equation has only a known zero witness.

Finally every non-hypothesis command decreases the floor, opens the new goal and hypothesis, and takes one common continuation. The prior context and goal cannot contain the fresh label meaningfully. Every successfully added context formula is an array. Hence a scalar goal cannot terminate successfully by exploiting the truth placeholder: it either rejects or continues consuming packets. These invariants ensure that every successful major has an intended formula shape.

6 A new obstruction to simultaneous restrictions

The named-specialization draft simulates arbitrary QE targets using equality naming and separately discusses restricted introduction witnesses. Whether both restrictions can be imposed together is a different question[3]. It has a negative answer for the native rules with no extra arbitrary-substitution rule.

Theorem 6.1 (Simultaneous restriction loses theorems)**.** *Restrict DI witnesses to zero or in-scope variables, and QE targets to one or in-scope variables. Then*

$$D_{x,y}(C(y,1,x)=1)$$

is unprovable, although it is a theorem of the unrestricted calculus. Adding any fixed finite set of numeral QE targets still leaves some unrestricted native theorems unprovable.

Proof of the first assertion. Evaluate numerical terms in the standard natural numbers, give every free variable value zero, and define a proof invariant by

$$[Q_yB]=[B(0)]\Rightarrow[B(1)],\qquad [D_{x,y}B]=[B(0,0)]\Rightarrow[B(0,1)].$$

An allowed DI witness has value zero; its fresh step parameter has value zero, so its successor has value one. Both DI modes preserve the displayed implication. DE's new closure has the same truth value as its major. Variable-target QE supplies the base, and target-one QE is implication elimination. Equality transport and computation preserve the invariant; known-zero-instance Supply preserves it by choosing its fresh parameter to be zero.

The displayed formula evaluates to true implying false: its zero instance is $C(0,1,0)=1$, while its one instance is $C(1,1,0)=1$. In the unrestricted calculus, case DI with witness 1 proves the formula immediately from $C(Sz,1,1)=1$. Appendix B gives the finite-range extension. □

This establishes failure of identity-language theorem equivalence for the specific simultaneous restrictions. It does not rule out all other interpretations, all restricted calculi, or all alternative domains. In particular an additional arbitrary-term substitution rule would change the system and invalidate the claimed restriction.

7 Verification, reproduction, and limits

The frozen compact and descriptive checkers passed all 144 complete-program certificate and malformed-input runs, with 3 successful targeted mutation controls. The 33 certificate cases were run on both programs under ordinary Python and `-O`; the remaining runs test malformed input. All 28 legacy cases are accounted for: 26 canonical translations, 24 already identical to replayed packets, and four additional successful CLI runs for the other two. The two obsolete successor-wrapper negatives were replaced explicitly rather than mislabeled under the new packet meaning.

The independent generator audit attempted 76,960 packets, obtained 57,640 successful generated equations, and checked 518,760 standard-model valuations at the fresh zero witness. It found no unequal endpoints and had no resource-bound skips. The compact source SHA-256 is

`bf17dcc00ea37d8e2e251c86b3ab53adca55ed403ca445bc22b9ee1564d5b540`.

The tests execute the complete standalone programs, including their input and verdict drivers, under ordinary and optimized Python. The examples exercise computation with used and unused state, transport, closure specialization, nonvacuous witnesses, compound case introduction, and contextual equation induction. Negative controls exercise failed majors, forbidden compound progress, future labels, malformed sources, and the false-equation exploit motivating the old eager guard. New constructor packets are checked against their new meaning, not assigned the semantics of an old packet with the same raw fields.

An independent finite numerical decoder evaluates zero instances of generated assumptions, including dummy slots and preexisting rigid parameters. These are falsification tests and reproducibility evidence. They do not prove the universal soundness, completeness, or bi-interpretation theorems; the written structural proofs establish those claims. The metatheory and Python runtime are not machine-verified.

The package contains the selected checker, two explicit profiles, a descriptive checker, an untrusted proof producer and converter, certificates, the unchanged source-measurement utility, source archives, hash manifests, test receipts, the complete LaTeX source, and this PDF. Run `python3 verify.py` for the finite reproduction and `python3 build.py` to rebuild the PDF and standalone TeX file. The original archives remain unchanged.

The construction retains the substantial inherited scalar-predecessor and equality-reflection arguments. It establishes a small strict implementation reduction, not a substantially simpler arithmetic foundation. The separate obstruction identifies a specific incorrect way to reduce the logical rules. Neither the source audit nor these proofs establish global minimality.

A Computation sources certified by construction

The computation packet has a mode field e and a triple $[p, a, r]$. The old packet interface already had an unused field in its computation branch; that field now carries e. The kernel generates the source counter itself. The relevant code is

```
n, t, r = s
h = [[u and [n], t, r], i(r, s, 1) if u else t]
```

Here `u` is the mode e, `s` is $[p, a, r]$, and `t` is reassigned to the initializer a. Canonical mode values are 0 for the base equation and 1 for the successor equation.

For base mode the generated equation is

$$I(0, a; v.r) = a.$$

The submitted predecessor payload p is unused in this case. For step mode the generated equation is

$$I(Sp, a; v.r) = r[I(p, a; v.r)/v].$$

The step counter is a singleton array because the kernel constructs `[n]`. It is never an untrusted purported successor. Thus no counter-shape validator is needed in this branch.

The expression `u and [n]` can return a noncanonical falsy mode unchanged. This causes no additional arithmetic case. If the subsequent common opening succeeds, a falsy numerical scalar or empty array denotes zero. Unsupported falsy JSON values reject when traversed. A truthy mode is used only as a branch tag and always produces the same singleton constructor. Its own JSON shape has no numerical meaning.

A.1 Demanded images

The opener validates an inserted image at numerical root whenever a meaningful exposed numerical coordinate requests that image. Consequently, if opening r demands the predecessor image $[p, a, r]$, the latter is checked before insertion. This validation is exactly the old eager check, performed at the first point where its result can affect the equation. Numerical-root validation is idempotent on successful

results: structural recursion leaves permitted coordinates unchanged, rejects a missing meaningful numerical image, and replaces only inessential formula leaves in checked annotations.

There is one apparent binding mismatch to check. In the constructed source, the displayed successor contributes an additional dummy zero slot to the environment for p. The predecessor image has only the iterator's usual two dummy slots. Root validation of the predecessor image excludes every meaningful reference to the extra outer slot. All genuine internal slots and existing rigid names therefore have the same meanings in both occurrences of p. The update's two aliased state slots are replaced by the validated predecessor image. After decoding, this gives precisely the scalar successor equation.

A.2 Unused images

If opening r never requests its image, r is independent of its outer state on every meaningful numerical path. Indeed, at opening depth d, references to internal binders are below d, while references to either removed state slot would request the image. Paths beneath a numerical binary annotation have no numerical meaning and do not affect this argument. Existing rigid names remain parameters.

The common continuation still opens the complete generated equation, and in particular traverses its source iterator. It therefore validates the submitted predecessor, initializer, and update in the environments where they actually occur. A malformed unused predecessor image is not itself a term used by the equation. Let w be the newly allocated parameter. After setting $w = 0$ and completing exposed outer slots by zero, write the source data as p_0, a_0, r_0. The generated equation is then

$$I(Sp_0, a_0; v.r_0) = r_0,$$

with r_0 independent of v. The scalar successor computation axiom proves it, because substitution of the unused recursive value changes nothing. Neither a validation nor a dummy-slot comparison of that unused raw recursive image is required.

A.3 The base equation and the fresh parameter

In base mode the source initializer occurs under two dummy zero slots, whereas its copy on the right may expose those coordinates during the common opening. The standard zero-extension argument applies: substitute zero for the new parameter, preserve genuine local binders and old rigid names, and compare the two finite syntax trees structurally. Their decoded initializers agree. The zero instance is therefore the base computation equation.

If the previous name floor is k_0, the new name is $w = k_0 - 1$. The original goal and retained assumptions were checked before this allocation, so they have no meaningful occurrence of w. Generated source data may acquire w at the later common opening. The three cases above establish a computation proof of the actual added equation's zero instance, $H(0)$. A successful continuation $\Gamma, H(w) \vdash A$ is discharged by renaming its later eigenparameters, substituting $w = 0$, and cutting with that computation proof. The old end sequent is unchanged. The rule never asserts that an arbitrary added equation $H(w)$ holds for every w.

A.4 Canonical completeness without zero aliases

Every ordinary base computation is represented by mode 0, its actual initializer and update, and an arbitrary unused predecessor payload (canonically zero). Every ordinary successor computation with predecessor p is represented by mode 1 and the triple $[p, a, r]$. In particular, a genuine zero predecessor

uses `n=0` together with `u=1`, and the generated source counter is exactly `[0]`. No truthy zero alias or additional equality proof is needed.

As in the inherited canonical emitter, active passive parameters are allocated rigid names, and dummy slots contain zero rather than references to otherwise unused enclosing slots. Thus inserting the displayed successor introduces no new dependency, and canonical opened updates have exactly their specified state substitutions. The result is the original computation equation in the original raw canonical syntax. Supplying it and immediately applying the hypothesis rule simulates an ordinary computation leaf. Other proof instructions and their completeness transformations are unchanged, apart from the already specified fresh-label accounting.

Precise scope. This is a new certificate format and a conservative computation generator. An old packet whose source counter was malformed is not being newly accepted with its old meaning. The new packet supplies a predecessor, and the checker constructs a genuine successor around it. Its actual generated equation is the one that must be used in the continuation. For example, a binary numerical predecessor denotes zero and yields a successor counter; it cannot trigger the false equation obtained by mistaking that binary node itself for a successor. The entire argument is about finite syntax and ordinary computation axioms on the full numerical domain. It does not use numeral testing as a substitute for the theorem, and does not claim an absolute smallest proof checker.

B A finite-range obstruction to simultaneous witness and specialization restriction

Let $\mathcal{C}$ be the native Choice–DI calculus with fixed scalar terms $0, S, I$, equations, binders $Q_y B$ and $D_{x,y} B$, and the rules specified in `ipra_named_specialization`. Consider the simultaneous restriction

$$\text{DI witnesses: } 0 \text{ or an in-scope variable}, \qquad \text{QE targets: } 1 \text{ or an in-scope variable}. \tag{R}$$

Neither arbitrary numerical substitution nor ordinary quantified logic is silently added as a primitive rule.

Theorem B.1. *Restriction (R) strictly decreases the native theorem set. In particular, with $C(y,p,q) = I(y,p;u.q)$ and $1 = S0$, the formula*

$$D_{x,y}(C(y,1,x) = 1) \tag{N1}$$

is a theorem of $\mathcal{C}$ but not of (R).

Proof. Evaluate numerical terms by their standard natural-number functions and assign zero to every free numerical variable. Define native formula truth recursively by ordinary equality and

$$[Q_y B] = [B(y := 0)] \Rightarrow [B(y := 1)],$$
$$[D_{x,y} B] = [B(x := 0, y := 0)] \Rightarrow [B(x := 0, y := 1)].$$

These clauses specify a proof invariant, not an ordinary first-order interpretation with unrestricted substitution. Formula substitution respects equality of the numerical values of substituted terms, since all binders are renamed away from those terms before substitution. Thus equality transport and the computation schemas preserve validity. Hypothesis use does so immediately.

An allowed DI witness has numerical value zero. Evaluating its fresh step parameter z at zero makes the successor Sz have value one. For case DI, the premise therefore supplies exactly the implication in the displayed D interpretation. For progress DI it supplies the same implication, because its current

hypothesis is evaluated at $z = 0$. The argument even applies to compound progress; the literal-equation restriction is not needed for this invariant. The retained context is unaffected by the fresh parameter.

In DE the fresh parameter a also has value zero. Consequently the added hypothesis $Q_y B(a, y)$ has exactly the truth value of its major $D_{x,y}B$. Validity of the continuation proves validity of the conclusion. A variable-target QE concludes the same truth value as its base premise; a target-one QE is ordinary implication elimination. All primitive rules preserve the invariant, so external induction on finite derivations establishes soundness.

In (N1) the zero instance is $C(0, 1, 0) = 1$, which is true, while the one instance is $C(1, 1, 0) = 1$, which says $0 = 1$ and is false. Hence (N1) is not derivable under (R). In the unrestricted calculus, case DI with witness 1 proves it immediately from the computation equation $C(Sz, 1, 1) = 1$, ignoring the base hypothesis. □

General finite-range version. Fix $N \geq 1$, let $W = \{0, \ldots, N\}$, allow QE only at variables and numerals in W, and continue to restrict DI witnesses to zero or variables. Evaluate free variables in W, keep ordinary numerical term semantics (whose outputs may lie outside W), and interpret

$$Q_y B := B(0) \Rightarrow \bigwedge_{j \in W} B(j), \qquad D_{x,y}B := \bigvee_{i \in W} \left(B(i, 0) \Rightarrow \bigwedge_{j \in W} B(i, j) \right).$$

A sequent is valid at every assignment of its free variables in W. Case DI is valid because each positive $j \leq N$ is $S(j - 1)$ with $j - 1 \in W$. Progress DI is valid by the finite external induction from zero to N. DE uses a witness in W and its continuation's validity at that assignment; QE uses a target in W. Equality transport is valid because the recursively defined formula semantics respects numerical equality even for substituted numerical values outside W. The same primitive computations hold. Thus all rules preserve validity. The unrestricted theorem

$$D_{x,y}\big(C(y, N + 1, x) = N + 1\big)$$

is false in this interpretation, since $1 \in W$ but no possible witness $i \in W$ equals $N + 1$. Adding finitely many allowed numeral targets therefore does not repair the simultaneous restriction.

Scope. This is an exact obstruction to identity-language theorem equivalence for these simultaneous restrictions. It is not a proof that every differently interpreted theory with restricted rules fails to be bi-interpretable with iPRA. The special proof invariant for (N1) is stronger than the $N = 1$ finite-range model and yields a smaller separating formula.

Compatibility with known-zero-instance Supply. The counterinterpretations remain sound for the rule

$$\frac{\Gamma, H(w) \vdash A \qquad \Gamma \vdash H(0)}{\Gamma \vdash A}, \qquad w \notin \mathrm{FV}(\Gamma, A).$$

In the fixed-zero invariant, $H(w)$ and $H(0)$ have the same truth value. For the finite-range invariant, extend any outer satisfying assignment by $w = 0$, which is permitted because $0 \in W$. The second premise makes $H(w)$ true at that extension; the first premise supplies A. Freshness makes the end sequent unaffected by that extension. This also covers the new lazy computation generator, whose separate soundness theorem proves its generated equation's zero instance. No arbitrary-term naming rule with a witness outside W is included.

The inherited bridge, reproduced with attribution

The next five sections are copied unchanged from the generated-assumptions archive at the pinned snapshot. That archive attributes the logical Choice-DI basis, scalar predecessor construction, actual-equality reflection, and primitive-recursion compiler to the earlier drafts. Their exact source paths and SHA-256 identities are in `evidence/source_manifest.json`. They are included so that the bi-interpretation does not depend on an unavailable external proof. None is claimed as a new result of this report. Their occurrences of ordinary logic describe the interpretation theory; they do not add primitives to the small native checker above.

C Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

C.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

C.2 Exact equivalence and the unused-witness construction

Lemma C.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem C.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition C.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

C.3 Induction and numerical cases separate cleanly

Lemma C.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma C.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

C.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

C.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem C.2 preserves every native sequent.

C.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [5] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [6]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion.

In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$

$$(Sx) + y = S(x + y), \tag{32}$$

$$d(Sx, Sy) = d(x, y), \tag{33}$$

$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [7]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section C give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

G Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u,\vec{p})$, introduce $i_h(n,a,\vec{p})$ with the two equations

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section C recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

G.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem G.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt: current draft collection*, commit `4a04139595eadc3e8818f1df2a94838ed4851d83`. https://github.com/VladimirReshetnikov/ProveIt/tree/4a04139595eadc3e8818f1df2a94838ed4851d83/docs/drafts.

[2] *Numerical axioms as local assumptions*, `ipra_generated_assumptions_2026-11-10.zip` in [1]. Source of the current leading generator, zero-witness discharge, scope decoder, test producer, and reproduced bridge.

[3] *Naming values before specialization*, `ipra_named_specialization_2026-11-10.zip` in [1]. Source of the named-specialization comparison and the question about simultaneous restrictions.

[4] *Endpoint certificates*, `ipra_endpoint_certificates_2026-11-09.zip` in [1]. Source of the shared whole-program measurement convention.

[5] Inherited scalar predecessor construction, preserved as `reference/scalar_predecessor.tex` in the generated-assumptions archive, with its earlier scalar-development attribution.

[6] Inherited actual-equality reflection construction, preserved as `reference/equality_reflection.tex` in the generated-assumptions archive, with attribution to the forward-left development.

[7] Inherited pairing and primitive-recursion compiler, preserved as `reference/primitive_recursion.tex` in the generated-assumptions archive, with attribution to the progress-closure development.

[8] Saeed Salehi. *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic*. Reports on Mathematical Logic **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.